

```

import {
  Document,
  Node,
  Plugin,
  ProcessOptions,
  Root,
  SourceMap,
  TransformCallback,
  Warning,
  WarningOptions
} from './postcss.js'
import Processor from './processor.js'

declare namespace Result {
  export interface Message {
    [others: string]: any

    /**
     * Source PostCSS plugin name.
     */
    plugin?: string

    /**
     * Message type.
     */
    type: string
  }
}

export interface ResultOptions extends ProcessOptions {
  /**
   * The CSS node that was the source of the warning.
   */
  node?: Node

  /**
   * Name of plugin that created this warning. `Result#warn` will fill it
   * automatically with `Plugin#postcssPlugin` value.
   */
  plugin?: string
}

export { Result_ as default }
}

/**
 * Provides the result of the PostCSS transformations.
 *
 * A Result instance is returned by `LazyResult#then`
 * or `Root#toResult` methods.
 *
 * ```js
 * postcss([autoprefixer]).process(css).then(result => {
 *   console.log(result.css)
 * })
 * ```
 *
 * ```js
 * const result2 = postcss.parse(css).toResult()

```

```

* ```
*/
declare class Result_<RootNode = Document | Root> {
  /**
   * A CSS string representing of `Result#root`.
   *
   * ```js
   * postcss.parse('a{}').toResult().css // => "a{}"
   * ```
   */
  css: string

  /**
   * Last runned PostCSS plugin.
   */
  lastPlugin: Plugin | TransformCallback

  /**
   * An instance of `SourceMapGenerator` class from the `source-map` library,
   * representing changes to the `Result#root` instance.
   *
   * ```js
   * result.map.toJSON() // => { version: 3, file: 'a.css', ... }
   * ```
   *
   * ```js
   * if (result.map) {
   *   fs.writeFileSync(result.opts.to + '.map', result.map.toString())
   * }
   * ```
   */
  map: SourceMap

  /**
   * Contains messages from plugins (e.g., warnings or custom messages).
   * Each message should have type and plugin properties.
   *
   * ```js
   * AtRule: {
   *   import: (atRule, { result }) {
   *     const importedFile = parseImport(atRule)
   *     result.messages.push({
   *       type: 'dependency',
   *       plugin: 'postcss-import',
   *       file: importedFile,
   *       parent: result.opts.from
   *     })
   *   }
   * }
   * ```
   */
  messages: Result.Message[]

  /**
   * Options from the `Processor#process` or `Root#toResult` call
   * that produced this Result instance.
   *
   * ```js

```

```

* root.toResult(opts).opts === opts
* ```
*/
opts: Result.ResultOptions

/**
 * The Processor instance used for this transformation.
 *
 * ```js
 * for (const plugin of result.processor.plugins) {
 *   if (plugin.postcssPlugin === 'postcss-bad') {
 *     throw 'postcss-good is incompatible with postcss-bad'
 *   }
 * }
 * ```
 */
processor: Processor

/**
 * Root node after all transformations.
 *
 * ```js
 * root.toResult().root === root
 * ```
 */
root: RootNode

/**
 * An alias for the `Result#css` property.
 * Use it with syntaxes that generate non-CSS output.
 *
 * ```js
 * result.css === result.content
 * ```
 */
get content(): string

/**
 * @param processor Processor used for this transformation.
 * @param root Root node after all transformations.
 * @param opts Options from the `Processor#process` or `Root#toResult`.
 */
constructor(processor: Processor, root: RootNode, opts: Result.ResultOptions)

/**
 * Returns for `Result#css` content.
 *
 * ```js
 * result + " === result.css
 * ```
 *
 * @return String representing of `Result#root`.
 */
toString(): string

/**
 * Creates an instance of `Warning` and adds it to `Result#messages`.
 *

```

```

* ```js
* if (decl.important) {
*   result.warn('Avoid !important', { node: decl, word: 'important' })
* }
* ```
*
* @param text Warning message.
* @param opts Warning options.
* @return Created warning.
*/
warn(message: string, options?: WarningOptions): Warning

/**
* Returns warnings from plugins. Filters `Warning` instances
* from `Result#messages`.
*
* ```js
* result.warnings().forEach(warn => {
*   console.warn(warn.toString())
* })
* ```
*
* @return Warnings from plugins.
*/
warnings(): Warning[]
}

declare class Result<RootNode = Document | Root> extends Result_<RootNode> {}

export = Result
import { shuffle } from "lodash";
export default shuffle;
import type { ImplicitLibVariableOptions } from '../variable';
export declare const decorators_legacy: Record<string, ImplicitLibVariableOptions>;
/** sourceMappingURL=decorators.legacy.d.ts.map
import { AjaxRequest } from './types';
**
* A normalized AJAX error.
*
* @see {@link ajax}
*/
export interface AjaxError extends Error {
  /**
   * The XHR instance associated with the error.
   */
  xhr: XMLHttpRequest;
  /**
   * The AjaxRequest associated with the error.
   */
  request: AjaxRequest;
  /**
   * The HTTP status code, if the request has completed. If not,
   * it is set to `0`.
   */
  status: number;
  /**
   * The responseType (e.g. 'json', 'arraybuffer', or 'xml').
   */

```

```

    responseType: XMLHttpRequestResponseType;
    /**
     * The response data.
     */
    response: any;
  }
  export interface AjaxErrorCtor {
    /**
     * @deprecated Internal implementation detail. Do not construct error instances.
     * Cannot be tagged as internal: https://github.com/ReactiveX/rxjs/issues/6269
     */
    new (message: string, xhr: XMLHttpRequest, request: AjaxRequest): AjaxError;
  }
  /**
   * Thrown when an error occurs during an AJAX request.
   * This is only exported because it is useful for checking to see if an error
   * is an `instanceof AjaxError`. DO NOT create new instances of `AjaxError` with
   * the constructor.
   *
   * @see {@link ajax}
   */
  export declare const AjaxError: AjaxErrorCtor;
  export interface AjaxTimeoutError extends AjaxError {
  }
  export interface AjaxTimeoutErrorCtor {
    /**
     * @deprecated Internal implementation detail. Do not construct error instances.
     * Cannot be tagged as internal: https://github.com/ReactiveX/rxjs/issues/6269
     */
    new (xhr: XMLHttpRequest, request: AjaxRequest): AjaxTimeoutError;
  }
  /**
   * Thrown when an AJAX request times out. Not to be confused with {@link TimeoutError}.
   *
   * This is exported only because it is useful for checking to see if errors are an
   * `instanceof AjaxTimeoutError`. DO NOT use the constructor to create an instance of
   * this type.
   *
   * @see {@link ajax}
   */
  export declare const AjaxTimeoutError: AjaxTimeoutErrorCtor;
  /**# sourceMappingURL=errors.d.ts.map
  import { Scene, Camera, ShaderMaterial, Vector2, MeshNormalMaterial, WebGLRenderTarget } from
  './../src/Three.js';

  import { Pass, FullScreenQuad } from './Pass.js';

  export interface RenderPixelatedPassParameters {
    normalEdgeStrength?: number;
    depthEdgeStrength?: number;
  }

  export class RenderPixelatedPass extends Pass {
    constructor(pixelSize: number, scene: Scene, camera: Camera, options?: RenderPixelatedPassParameters);
    pixelSize: number;
    resolution: Vector2;
    renderResolution: Vector2;
  }

```

```

    pixelatedMaterial: ShaderMaterial;
    normalMaterial: MeshNormalMaterial;

    fsQuad: FullScreenQuad;
    scene: Scene;
    camera: Camera;

    normalEdgeStrength: RenderPixelatedPassParameters['normalEdgeStrength'];
    depthEdgeStrength: RenderPixelatedPassParameters['depthEdgeStrength'];

    beautyRenderTarget: WebGLRenderTarget;
    normalRenderTarget: WebGLRenderTarget;

    setPixelSize(pixelSize: number): void;
}
/**
 * 曲线辅助模块
 */

import {
    create as v2Create,
    distSquare as v2DistSquare,
    VectorArray
} from './vector';

const mathPow = Math.pow;
const mathSqrt = Math.sqrt;

const EPSILON = 1e-8;
const EPSILON_NUMERIC = 1e-4;

const THREE_SQRT = mathSqrt(3);
const ONE_THIRD = 1 / 3;

// 临时变量
const _v0 = v2Create();
const _v1 = v2Create();
const _v2 = v2Create();

function isAroundZero(val: number) {
    return val > -EPSILON && val < EPSILON;
}
function isNotAroundZero(val: number) {
    return val > EPSILON || val < -EPSILON;
}
/**
 * 计算三次贝塞尔值
 */
export function cubicAt(p0: number, p1: number, p2: number, p3: number, t: number): number {
    const onet = 1 - t;
    return onet * onet * (onet * p0 + 3 * t * p1)
        + t * t * (t * p3 + 3 * onet * p2);
}

/**
 * 计算三次贝塞尔导数值
 */

```

```

export function cubicDerivativeAt(p0: number, p1: number, p2: number, p3: number, t: number): number {
  const onet = 1 - t;
  return 3 * (
    ((p1 - p0) * onet + 2 * (p2 - p1) * t) * onet
    + (p3 - p2) * t * t
  );
}

/**
 * 计算三次贝塞尔方程根，使用盛金公式
 */
export function cubicRootAt(p0: number, p1: number, p2: number, p3: number, val: number, roots: number[]): number {
  // Evaluate roots of cubic functions
  const a = p3 + 3 * (p1 - p2) - p0;
  const b = 3 * (p2 - p1 * 2 + p0);
  const c = 3 * (p1 - p0);
  const d = p0 - val;

  const A = b * b - 3 * a * c;
  const B = b * c - 9 * a * d;
  const C = c * c - 3 * b * d;

  let n = 0;

  if (isAroundZero(A) && isAroundZero(B)) {
    if (isAroundZero(b)) {
      roots[0] = 0;
    }
    else {
      const t1 = -c / b; //t1, t2, t3, b is not zero
      if (t1 >= 0 && t1 <= 1) {
        roots[n++] = t1;
      }
    }
  }
  else {
    const disc = B * B - 4 * A * C;

    if (isAroundZero(disc)) {
      const K = B / A;
      const t1 = -b / a + K; // t1, a is not zero
      const t2 = -K / 2; // t2, t3
      if (t1 >= 0 && t1 <= 1) {
        roots[n++] = t1;
      }
      if (t2 >= 0 && t2 <= 1) {
        roots[n++] = t2;
      }
    }
    else if (disc > 0) {
      const discSqrt = mathSqrt(disc);
      let Y1 = A * b + 1.5 * a * (-B + discSqrt);
      let Y2 = A * b + 1.5 * a * (-B - discSqrt);
      if (Y1 < 0) {
        Y1 = -mathPow(-Y1, ONE_THIRD);
      }
      else {
        Y1 = mathPow(Y1, ONE_THIRD);
      }
    }
  }
}

```

```

    }
    if (Y2 < 0) {
        Y2 = -mathPow(-Y2, ONE_THIRD);
    }
    else {
        Y2 = mathPow(Y2, ONE_THIRD);
    }
    const t1 = (-b - (Y1 + Y2)) / (3 * a);
    if (t1 >= 0 && t1 <= 1) {
        roots[n++] = t1;
    }
}
else {
    const T = (2 * A * b - 3 * a * B) / (2 * mathSqrt(A * A * A));
    const theta = Math.acos(T) / 3;
    const ASqrt = mathSqrt(A);
    const tmp = Math.cos(theta);

    const t1 = (-b - 2 * ASqrt * tmp) / (3 * a);
    const t2 = (-b + ASqrt * (tmp + THREE_SQRT * Math.sin(theta))) / (3 * a);
    const t3 = (-b + ASqrt * (tmp - THREE_SQRT * Math.sin(theta))) / (3 * a);
    if (t1 >= 0 && t1 <= 1) {
        roots[n++] = t1;
    }
    if (t2 >= 0 && t2 <= 1) {
        roots[n++] = t2;
    }
    if (t3 >= 0 && t3 <= 1) {
        roots[n++] = t3;
    }
}
}
return n;
}

/**
 * 计算三次贝塞尔方程极限值的位置
 * @return 有效数目
 */
export function cubicExtrema(p0: number, p1: number, p2: number, p3: number, extrema: number[]): number {
    const b = 6 * p2 - 12 * p1 + 6 * p0;
    const a = 9 * p1 + 3 * p3 - 3 * p0 - 9 * p2;
    const c = 3 * p1 - 3 * p0;

    let n = 0;
    if (isAroundZero(a)) {
        if (isNotAroundZero(b)) {
            const t1 = -c / b;
            if (t1 >= 0 && t1 <= 1) {
                extrema[n++] = t1;
            }
        }
    }
    else {
        const disc = b * b - 4 * a * c;
        if (isAroundZero(disc)) {
            extrema[0] = -b / (2 * a);
        }
    }
}

```



```

    else if (disc > 0) {
        const discSqrt = mathSqrt(disc);
        const t1 = (-b + discSqrt) / (2 * a);
        const t2 = (-b - discSqrt) / (2 * a);
        if (t1 >= 0 && t1 <= 1) {
            extrema[n++] = t1;
        }
        if (t2 >= 0 && t2 <= 1) {
            extrema[n++] = t2;
        }
    }
}
return n;
}

/**
 * 细分三次贝塞尔曲线
 */
export function cubicSubdivide(p0: number, p1: number, p2: number, p3: number, t: number, out: number[]) {
    const p01 = (p1 - p0) * t + p0;
    const p12 = (p2 - p1) * t + p1;
    const p23 = (p3 - p2) * t + p2;

    const p012 = (p12 - p01) * t + p01;
    const p123 = (p23 - p12) * t + p12;

    const p0123 = (p123 - p012) * t + p012;
    // Seg0
    out[0] = p0;
    out[1] = p01;
    out[2] = p012;
    out[3] = p0123;
    // Seg1
    out[4] = p0123;
    out[5] = p123;
    out[6] = p23;
    out[7] = p3;
}

/**
 * 投射点到三次贝塞尔曲线上，返回投射距离。
 * 投射点有可能会有一个或者多个，这里只返回其中距离最短的一个。
 */
export function cubicProjectPoint(
    x0: number, y0: number, x1: number, y1: number, x2: number, y2: number, x3: number, y3: number,
    x: number, y: number, out: VectorArray
): number {
    // http://pomax.github.io/bezierinfo/#projections
    let t;
    let interval = 0.005;
    let d = Infinity;
    let prev;
    let next;
    let d1;
    let d2;

    _v0[0] = x;
    _v0[1] = y;

```

```

// 先粗略估计一下可能的最小距离的 t 值
// PENDING
for (let _t = 0; _t < 1; _t += 0.05) {
    _v1[0] = cubicAt(x0, x1, x2, x3, _t);
    _v1[1] = cubicAt(y0, y1, y2, y3, _t);
    d1 = v2DistSquare(_v0, _v1);
    if (d1 < d) {
        t = _t;
        d = d1;
    }
}
d = Infinity;

// At most 32 iteration
for (let i = 0; i < 32; i++) {
    if (interval < EPSILON_NUMERIC) {
        break;
    }
    prev = t - interval;
    next = t + interval;
    // t - interval
    _v1[0] = cubicAt(x0, x1, x2, x3, prev);
    _v1[1] = cubicAt(y0, y1, y2, y3, prev);

    d1 = v2DistSquare(_v1, _v0);

    if (prev >= 0 && d1 < d) {
        t = prev;
        d = d1;
    }
    else {
        // t + interval
        _v2[0] = cubicAt(x0, x1, x2, x3, next);
        _v2[1] = cubicAt(y0, y1, y2, y3, next);
        d2 = v2DistSquare(_v2, _v0);

        if (next <= 1 && d2 < d) {
            t = next;
            d = d2;
        }
        else {
            interval *= 0.5;
        }
    }
}
// t
if (out) {
    out[0] = cubicAt(x0, x1, x2, x3, t);
    out[1] = cubicAt(y0, y1, y2, y3, t);
}
// console.log(interval, i);
return mathSqrt(d);
}

/**
 * 计算三次贝塞尔曲线长度
 */

```

```

export function cubicLength(
  x0: number, y0: number, x1: number, y1: number, x2: number, y2: number, x3: number, y3: number,
  iteration: number
){
  let px = x0;
  let py = y0;

  let d = 0;

  const step = 1 / iteration;

  for (let i = 1; i <= iteration; i++) {
    let t = i * step;
    const x = cubicAt(x0, x1, x2, x3, t);
    const y = cubicAt(y0, y1, y2, y3, t);

    const dx = x - px;
    const dy = y - py;

    d += Math.sqrt(dx * dx + dy * dy);

    px = x;
    py = y;
  }

  return d;
}

/**
 * 计算二次方贝塞尔值
 */
export function quadraticAt(p0: number, p1: number, p2: number, t: number): number {
  const onet = 1 - t;
  return onet * (onet * p0 + 2 * t * p1) + t * t * p2;
}

/**
 * 计算二次方贝塞尔导数值
 */
export function quadraticDerivativeAt(p0: number, p1: number, p2: number, t: number): number {
  return 2 * ((1 - t) * (p1 - p0) + t * (p2 - p1));
}

/**
 * 计算二次方贝塞尔方程根
 * @return 有效根数目
 */
export function quadraticRootAt(p0: number, p1: number, p2: number, val: number, roots: number[]): number {
  const a = p0 - 2 * p1 + p2;
  const b = 2 * (p1 - p0);
  const c = p0 - val;

  let n = 0;
  if (isAroundZero(a)) {
    if (isNotAroundZero(b)) {
      const t1 = -c / b;
      if (t1 >= 0 && t1 <= 1) {

```

```

        roots[n++] = t1;
    }
}
}
else {
    const disc = b * b - 4 * a * c;
    if (isAroundZero(disc)) {
        const t1 = -b / (2 * a);
        if (t1 >= 0 && t1 <= 1) {
            roots[n++] = t1;
        }
    }
    else if (disc > 0) {
        const discSqrt = mathSqrt(disc);
        const t1 = (-b + discSqrt) / (2 * a);
        const t2 = (-b - discSqrt) / (2 * a);
        if (t1 >= 0 && t1 <= 1) {
            roots[n++] = t1;
        }
        if (t2 >= 0 && t2 <= 1) {
            roots[n++] = t2;
        }
    }
}
}
return n;
}

/**
 * 计算二次贝塞尔方程极限值
 */
export function quadraticExtremum(p0: number, p1: number, p2: number): number {
    const divider = p0 + p2 - 2 * p1;
    if (divider === 0) {
        // p1 is center of p0 and p2
        return 0.5;
    }
    else {
        return (p0 - p1) / divider;
    }
}

/**
 * 细分二次贝塞尔曲线
 */
export function quadraticSubdivide(p0: number, p1: number, p2: number, t: number, out: number[]) {
    const p01 = (p1 - p0) * t + p0;
    const p12 = (p2 - p1) * t + p1;
    const p012 = (p12 - p01) * t + p01;

    // Seg0
    out[0] = p0;
    out[1] = p01;
    out[2] = p012;

    // Seg1
    out[3] = p012;
    out[4] = p12;
    out[5] = p2;
}

```

```

}

/**
 * 投射点到二次贝塞尔曲线上，返回投射距离。
 * 投射点有可能会有一个或者多个，这里只返回其中距离最短的一个。
 * @param {number} x0
 * @param {number} y0
 * @param {number} x1
 * @param {number} y1
 * @param {number} x2
 * @param {number} y2
 * @param {number} x
 * @param {number} y
 * @param {Array.<number>} out 投射点
 * @return {number}
 */
export function quadraticProjectPoint(
  x0: number, y0: number, x1: number, y1: number, x2: number, y2: number,
  x: number, y: number, out: VectorArray
): number {
  // http://pomax.github.io/bezierinfo/#projections
  let t: number;
  let interval = 0.005;
  let d = Infinity;

  _v0[0] = x;
  _v0[1] = y;

  // 先粗略估计一下可能的最小距离的 t 值
  // PENDING
  for (let _t = 0; _t < 1; _t += 0.05) {
    _v1[0] = quadraticAt(x0, x1, x2, _t);
    _v1[1] = quadraticAt(y0, y1, y2, _t);
    const d1 = v2DistSquare(_v0, _v1);
    if (d1 < d) {
      t = _t;
      d = d1;
    }
  }
  d = Infinity;

  // At most 32 iteration
  for (let i = 0; i < 32; i++) {
    if (interval < EPSILON_NUMERIC) {
      break;
    }
    const prev = t - interval;
    const next = t + interval;
    // t - interval
    _v1[0] = quadraticAt(x0, x1, x2, prev);
    _v1[1] = quadraticAt(y0, y1, y2, prev);

    const d1 = v2DistSquare(_v1, _v0);

    if (prev >= 0 && d1 < d) {
      t = prev;
      d = d1;
    }
  }
}

```

```

    }
    else {
        // t + interval
        _v2[0] = quadraticAt(x0, x1, x2, next);
        _v2[1] = quadraticAt(y0, y1, y2, next);
        const d2 = v2DistSquare(_v2, _v0);
        if (next <= 1 && d2 < d) {
            t = next;
            d = d2;
        }
        else {
            interval *= 0.5;
        }
    }
}
// t
if (out) {
    out[0] = quadraticAt(x0, x1, x2, t);
    out[1] = quadraticAt(y0, y1, y2, t);
}
// console.log(interval, i);
return mathSqrt(d);
}

/**
 * 计算二次贝塞尔曲线长度
 */
export function quadraticLength(
    x0: number, y0: number, x1: number, y1: number, x2: number, y2: number,
    iteration: number
) {
    let px = x0;
    let py = y0;

    let d = 0;

    const step = 1 / iteration;

    for (let i = 1; i <= iteration; i++) {
        let t = i * step;
        const x = quadraticAt(x0, x1, x2, t);
        const y = quadraticAt(y0, y1, y2, t);

        const dx = x - px;
        const dy = y - py;

        d += Math.sqrt(dx * dx + dy * dy);

        px = x;
        py = y;
    }

    return d;
}

import Axis from '../Axis.js';
import Scale from '../scale/Scale.js';
import { OptionAxisType } from './axisCommonTypes.js';
import SingleAxisModel, { SingleAxisPosition } from './AxisModel.js';

```

```

import { LayoutOrient } from '../util/types.js';
import Single from './Single.js';
interface SingleAxis {
  /**
   * Transform global coord to local coord,
   * i.e. let localCoord = axis.toLocalCoord(80);
   */
  toLocalCoord(coord: number): number;
  /**
   * Transform global coord to local coord,
   * i.e. let globalCoord = axis.toLocalCoord(40);
   */
  toGlobalCoord(coord: number): number;
}
declare class SingleAxis extends Axis {
  position: SingleAxisPosition;
  orient: LayoutOrient;
  coordinateSystem: Single;
  model: SingleAxisModel;
  constructor(dim: string, scale: Scale, coordExtent: [number, number], axisType?: OptionAxisType, position?: SingleAxisPosition);
  /**
   * Judge the orient of the axis.
   */
  isHorizontal(): boolean;
  pointToData(point: number[], clamp?: boolean): number;
}
export default SingleAxis;
declare function getDunderProto(target: {}): object | null;

declare const x: false | typeof getDunderProto;

export = x;
import {
  type IssueData,
  type StringValidation,
  type ZodCustomIssue,
  ZodError,
  type ZodErrorMap,
  type ZodIssue,
  ZodIssueCode,
} from './ZodError.js';
import { defaultErrorMap, getErrorMap } from './errors.js';
import type { enumUtil } from './helpers/enumUtil.js';
import { errorUtil } from './helpers/errorUtil.js';
import {
  type AsyncParseReturnType,
  DIRTY,
  INVALID,
  OK,
  type ParseContext,
  type ParseInput,
  type ParseParams,
  type ParsePath,
  type ParseReturnType,
  ParseStatus,
  type SyncParseReturnType,
  addIssueToContext,

```

```

    isAborted,
    isAsync,
    isDirty,
    isValid,
    makeIssue,
  } from "./helpers/parseUtil.js";
import type { partialUtil } from "./helpers/partialUtil.js";
import type { Primitive } from "./helpers/typeAliases.js";
import { util, ZodParsedType, getParsedType, type objectUtil } from "./helpers/util.js";
import type { StandardSchemaV1 } from "./standard-schema.js";

//////////
//////////
//////////      //////////
//////////    ZodType  //////////
//////////      //////////
//////////
//////////
//////////

export interface RefinementCtx {
  addIssue: (arg: IssueData) => void;
  path: (string | number)[];
}
export type ZodRawShape = { [k: string]: ZodTypeAny };
export type ZodTypeAny = ZodType<any, any, any>;
export type TypeOf<T extends ZodType<any, any, any>> = T["_output"];
export type input<T extends ZodType<any, any, any>> = T["_input"];
export type output<T extends ZodType<any, any, any>> = T["_output"];
export type { TypeOf as infer };

export type CustomErrorParams = Partial<util.Omit<ZodCustomIssue, "code">>>;
export interface ZodTypeDef {
  errorMap?: ZodErrorMap | undefined;
  description?: string | undefined;
}

class ParseInputLazyPath implements ParseInput {
  parent: ParseContext;
  data: any;
  _path: ParsePath;
  _key: string | number | (string | number)[];
  _cachedPath: ParsePath = [];
  constructor(parent: ParseContext, value: any, path: ParsePath, key: string | number | (string | number)[]) {
    this.parent = parent;
    this.data = value;
    this._path = path;
    this._key = key;
  }
  get path() {
    if (!this._cachedPath.length) {
      if (Array.isArray(this._key)) {
        this._cachedPath.push(...this._path, ...this._key);
      } else {
        this._cachedPath.push(...this._path, this._key);
      }
    }
  }

  return this._cachedPath;

```



```

    }
  }

const handleResult = <Input, Output>(
  ctx: ParseContext,
  result: SyncParseReturnType<Output>
): { success: true; data: Output } | { success: false; error: ZodError<Input> } => {
  if (isValid(result)) {
    return { success: true, data: result.value };
  } else {
    if (!ctx.common.issues.length) {
      throw new Error("Validation failed but no issues detected.");
    }

    return {
      success: false,
      get error() {
        if ((this as any)._error) return (this as any)._error as Error;
        const error = new ZodError(ctx.common.issues);
        (this as any)._error = error;
        return (this as any)._error;
      },
    };
  }
};

export type RawCreateParams =
  | {
    errorMap?: ZodErrorMap | undefined;
    invalid_type_error?: string | undefined;
    required_error?: string | undefined;
    message?: string | undefined;
    description?: string | undefined;
  }
  | undefined;
export type ProcessedCreateParams = {
  errorMap?: ZodErrorMap | undefined;
  description?: string | undefined;
};
function processCreateParams(params: RawCreateParams): ProcessedCreateParams {
  if (!params) return {};
  const { errorMap, invalid_type_error, required_error, description } = params;
  if (errorMap && (invalid_type_error || required_error)) {
    throw new Error(`Can't use "invalid_type_error" or "required_error" in conjunction with custom error map.`);
  }
  if (errorMap) return { errorMap: errorMap, description };
  const customMap: ZodErrorMap = (iss, ctx) => {
    const { message } = params;

    if (iss.code === "invalid_enum_value") {
      return { message: message ?? ctx.defaultError };
    }
    if (typeof ctx.data === "undefined") {
      return { message: message ?? required_error ?? ctx.defaultError };
    }
    if (iss.code !== "invalid_type") return { message: ctx.defaultError };
    return { message: message ?? invalid_type_error ?? ctx.defaultError };
  };
};

```

```

    return { errorMap: customMap, description };
  }

export type SafeParseSuccess<Output> = {
  success: true;
  data: Output;
  error?: never;
};
export type SafeParseError<Input> = {
  success: false;
  error: ZodError<Input>;
  data?: never;
};

export type SafeParseReturnType<Input, Output> = SafeParseSuccess<Output> | SafeParseError<Input>;

export abstract class ZodType<Output = any, Def extends ZodTypeDef = ZodTypeDef, Input = Output> {
  readonly _type!: Output;
  readonly _output!: Output;
  readonly _input!: Input;
  readonly _def!: Def;

  get description(): string | undefined {
    return this._def.description;
  }

  "~standard": StandardSchemaV1.Props<Input, Output>;

  abstract _parse(input: ParseInput): ParseReturnType<Output>;

  _getType(input: ParseInput): string {
    return getParsedType(input.data);
  }

  _getOrReturnCtx(input: ParseInput, ctx?: ParseContext | undefined): ParseContext {
    return (
      ctx || {
        common: input.parent.common,
        data: input.data,

        parsedType: getParsedType(input.data),

        schemaErrorMap: this._def.errorMap,
        path: input.path,
        parent: input.parent,
      }
    );
  }

  _processInputParams(input: ParseInput): {
    status: ParseStatus;
    ctx: ParseContext;
  } {
    return {
      status: new ParseStatus(),
      ctx: {
        common: input.parent.common,
        data: input.data,
      },
    };
  }
}

```

```

    parsedType: getParsedType(input.data),

    schemaErrorMap: this._def.errorMap,
    path: input.path,
    parent: input.parent,
  },
};
}

_parseSync(input: ParseInput): SyncParseReturnType<Output> {
  const result = this._parse(input);
  if (isAsync(result)) {
    throw new Error("Synchronous parse encountered promise.");
  }
  return result;
}

_parseAsync(input: ParseInput): AsyncParseReturnType<Output> {
  const result = this._parse(input);
  return Promise.resolve(result);
}

parse(data: unknown, params?: util.InexactPartial<ParseParams>): Output {
  const result = this.safeParse(data, params);
  if (result.success) return result.data;
  throw result.error;
}

safeParse(data: unknown, params?: util.InexactPartial<ParseParams>): SafeParseReturnType<Input, Output> {
  const ctx: ParseContext = {
    common: {
      issues: [],
      async: params?.async ?? false,
      contextualErrorMap: params?.errorMap,
    },
    path: params?.path || [],
    schemaErrorMap: this._def.errorMap,
    parent: null,
    data,
    parsedType: getParsedType(data),
  };
  const result = this._parseSync({ data, path: ctx.path, parent: ctx });

  return handleResult(ctx, result);
}

"~validate"(data: unknown): StandardSchemaV1.Result<Output> | Promise<StandardSchemaV1.Result<Output>> {
  const ctx: ParseContext = {
    common: {
      issues: [],
      async: !(this["~standard"] as any).async,
    },
    path: [],
    schemaErrorMap: this._def.errorMap,
    parent: null,
    data,
    parsedType: getParsedType(data),
  };

```

```

};

if (!this["~standard"] as any).async) {
  try {
    const result = this._parseSync({ data, path: [], parent: ctx });
    return isValid(result)
      ? {
          value: result.value,
        }
      : {
          issues: ctx.common.issues,
        };
  } catch (err: any) {
    if ((err as Error)?.message?.toLowerCase()?.includes("encountered")) {
      (this["~standard"] as any).async = true;
    }
    (ctx as any).common = {
      issues: [],
      async: true,
    };
  }
}

return this._parseAsync({ data, path: [], parent: ctx }).then((result) =>
  isValid(result)
    ? {
        value: result.value,
      }
    : {
        issues: ctx.common.issues,
      }
  );
}

async parseAsync(data: unknown, params?: util.InexactPartial<ParseParams>): Promise<Output> {
  const result = await this.safeParseAsync(data, params);
  if (result.success) return result.data;
  throw result.error;
}

async safeParseAsync(
  data: unknown,
  params?: util.InexactPartial<ParseParams>
): Promise<SafeParseReturnType<Input, Output>> {
  const ctx: ParseContext = {
    common: {
      issues: [],
      contextualErrorMap: params?.errorMap,
      async: true,
    },
    path: params?.path || [],
    schemaErrorMap: this._def.errorMap,
    parent: null,
    data,
    parsedType: getParsedType(data),
  };
  const maybeAsyncResult = this._parse({ data, path: ctx.path, parent: ctx });

```

```

    const result = await (isAsync(maybeAsyncResult) ? maybeAsyncResult : Promise.resolve(maybeAsyncResult));
    return handleResult(ctx, result);
  }

  /** Alias of safeParseAsync */
  spa = this.safeParseAsync;

  refine<RefinedOutput extends Output>(
    check: (arg: Output) => arg is RefinedOutput,
    message?: string | CustomErrorParams | ((arg: Output) => CustomErrorParams)
  ): ZodEffects<this, RefinedOutput, Input>;
  refine(
    check: (arg: Output) => unknown | Promise<unknown>,
    message?: string | CustomErrorParams | ((arg: Output) => CustomErrorParams)
  ): ZodEffects<this, Output, Input>;
  refine(
    check: (arg: Output) => unknown,
    message?: string | CustomErrorParams | ((arg: Output) => CustomErrorParams)
  ): ZodEffects<this, Output, Input> {
    const getIssueProperties = (val: Output) => {
      if (typeof message === "string" || typeof message === "undefined") {
        return { message };
      } else if (typeof message === "function") {
        return message(val);
      } else {
        return message;
      }
    };
  };
  return this._refinement((val, ctx) => {
    const result = check(val);
    const setError = () =>
      ctx.addIssue({
        code: ZodIssueCode.custom,
        ...getIssueProperties(val),
      });
    if (typeof Promise !== "undefined" && result instanceof Promise) {
      return result.then((data) => {
        if (!data) {
          setError();
          return false;
        } else {
          return true;
        }
      });
    }
    if (!result) {
      setError();
      return false;
    } else {
      return true;
    }
  });
}

refinement<RefinedOutput extends Output>(
  check: (arg: Output) => arg is RefinedOutput,
  refinementData: IssueData | ((arg: Output, ctx: RefinementCtx) => IssueData)
): ZodEffects<this, RefinedOutput, Input>;

```

```

refinement(
  check: (arg: Output) => boolean,
  refinementData: IssueData | ((arg: Output, ctx: RefinementCtx) => IssueData)
): ZodEffects<this, Output, Input>;
refinement(
  check: (arg: Output) => unknown,
  refinementData: IssueData | ((arg: Output, ctx: RefinementCtx) => IssueData)
): ZodEffects<this, Output, Input> {
  return this._refinement((val, ctx) => {
    if (!check(val)) {
      ctx.addIssue(typeof refinementData === "function" ? refinementData(val, ctx) : refinementData);
      return false;
    } else {
      return true;
    }
  });
}

_refinement(refinement: RefinementEffect<Output>["refinement"]): ZodEffects<this, Output, Input> {
  return new ZodEffects({
    schema: this,
    typeName: ZodFirstPartyTypeKind.ZodEffects,
    effect: { type: "refinement", refinement },
  });
}

superRefine<RefinedOutput extends Output>({
  refinement: (arg: Output, ctx: RefinementCtx) => arg is RefinedOutput
}): ZodEffects<this, RefinedOutput, Input>;
superRefine(refinement: (arg: Output, ctx: RefinementCtx) => void): ZodEffects<this, Output, Input>;
superRefine(refinement: (arg: Output, ctx: RefinementCtx) => Promise<void>): ZodEffects<this, Output, Input>;
superRefine(
  refinement: (arg: Output, ctx: RefinementCtx) => unknown | Promise<unknown>
): ZodEffects<this, Output, Input> {
  return this._refinement(refinement);
}

constructor(def: Def) {
  this._def = def;
  this.parse = this.parse.bind(this);
  this.safeParse = this.safeParse.bind(this);
  this.parseAsync = this.parseAsync.bind(this);
  this.safeParseAsync = this.safeParseAsync.bind(this);
  this.spa = this.spa.bind(this);
  this.refine = this.refine.bind(this);
  this.refinement = this.refinement.bind(this);
  this.superRefine = this.superRefine.bind(this);
  this.optional = this.optional.bind(this);
  this.nullable = this.nullable.bind(this);
  this.nullish = this.nullish.bind(this);
  this.array = this.array.bind(this);
  this.promise = this.promise.bind(this);
  this.or = this.or.bind(this);
  this.and = this.and.bind(this);
  this.transform = this.transform.bind(this);
  this.brand = this.brand.bind(this);
  this.default = this.default.bind(this);
  this.catch = this.catch.bind(this);
}

```

```

    this.describe = this.describe.bind(this);
    this.pipe = this.pipe.bind(this);
    this.readonly = this.readonly.bind(this);
    this.isNullable = this.isNullable.bind(this);
    this.isOptional = this.isOptional.bind(this);
    this["~standard"] = {
      version: 1,
      vendor: "zod",
      validate: (data) => this["~validate"](data),
    };
  }

  optional(): ZodOptional<this> {
    return ZodOptional.create(this, this._def) as any;
  }
  nullable(): ZodNullable<this> {
    return ZodNullable.create(this, this._def) as any;
  }
  nullish(): ZodOptional<ZodNullable<this>> {
    return this.nullable().optional();
  }
  array(): ZodArray<this> {
    return ZodArray.create(this);
  }
  promise(): ZodPromise<this> {
    return ZodPromise.create(this, this._def);
  }

  or<T extends ZodTypeAny>(option: T): ZodUnion<[this, T]> {
    return ZodUnion.create([this, option], this._def) as any;
  }

  and<T extends ZodTypeAny>(incoming: T): ZodIntersection<this, T> {
    return ZodIntersection.create(this, incoming, this._def);
  }

  transform<NewOut>(
    transform: (arg: Output, ctx: RefinementCtx) => NewOut | Promise<NewOut>
  ): ZodEffects<this, NewOut> {
    return new ZodEffects({
      ...processCreateParams(this._def),
      schema: this,
      typeName: ZodFirstPartyTypeKind.ZodEffects,
      effect: { type: "transform", transform },
    }) as any;
  }

  default(def: util.noUndefined<Input>): ZodDefault<this>;
  default(def: () => util.noUndefined<Input>): ZodDefault<this>;
  default(def: any) {
    const defaultValueFunc = typeof def === "function" ? def : () => def;

    return new ZodDefault({
      ...processCreateParams(this._def),
      innerType: this,
      defaultValue: defaultValueFunc,
      typeName: ZodFirstPartyTypeKind.ZodDefault,
    }) as any;
  }

```

```

}

brand<B extends string | number | symbol>(brand?: B): ZodBranded<this, B>;
brand<B extends string | number | symbol>(): ZodBranded<this, B> {
  return new ZodBranded({
    typeName: ZodFirstPartyTypeKind.ZodBranded,
    type: this,
    ...processCreateParams(this._def),
  });
}

catch(def: Output): ZodCatch<this>;
catch(def: (ctx: { error: ZodError; input: Input }) => Output): ZodCatch<this>;
catch(def: any) {
  const catchValueFunc = typeof def === "function" ? def : () => def;

  return new ZodCatch({
    ...processCreateParams(this._def),
    innerType: this,
    catchValue: catchValueFunc,
    typeName: ZodFirstPartyTypeKind.ZodCatch,
  }) as any;
}

describe(description: string): this {
  const This = (this as any).constructor;
  return new This({
    ...this._def,
    description,
  });
}

pipe<T extends ZodTypeAny>(target: T): ZodPipeline<this, T> {
  return ZodPipeline.create(this, target);
}

readonly(): ZodReadonly<this> {
  return ZodReadonly.create(this);
}

isOptional(): boolean {
  return this.safeParse(undefined).success;
}

isNullable(): boolean {
  return this.safeParse(null).success;
}
}

////////////////////////////////////
////////////////////////////////////
//////////          //////////
//////////  ZodString  //////////
//////////          //////////
////////////////////////////////////
////////////////////////////////////
export type IpVersion = "v4" | "v6";
export type ZodStringCheck =
  | { kind: "min"; value: number; message?: string | undefined }
  | { kind: "max"; value: number; message?: string | undefined }

```



```

| { kind: "length"; value: number; message?: string | undefined }
| { kind: "email"; message?: string | undefined }
| { kind: "url"; message?: string | undefined }
| { kind: "emoji"; message?: string | undefined }
| { kind: "uuid"; message?: string | undefined }
| { kind: "nanoid"; message?: string | undefined }
| { kind: "cuid"; message?: string | undefined }
| { kind: "includes"; value: string; position?: number | undefined; message?: string | undefined }
| { kind: "cuid2"; message?: string | undefined }
| { kind: "ulid"; message?: string | undefined }
| { kind: "startsWith"; value: string; message?: string | undefined }
| { kind: "endsWith"; value: string; message?: string | undefined }
| { kind: "regex"; regex: RegExp; message?: string | undefined }
| { kind: "trim"; message?: string | undefined }
| { kind: "toLowerCase"; message?: string | undefined }
| { kind: "toUpperCase"; message?: string | undefined }
| { kind: "jwt"; alg?: string; message?: string | undefined }
| {
  kind: "datetime";
  offset: boolean;
  local: boolean;
  precision: number | null;
  message?: string | undefined;
}
| {
  kind: "date";
  // withDate: true;
  message?: string | undefined;
}
| {
  kind: "time";
  precision: number | null;
  message?: string | undefined;
}
| { kind: "duration"; message?: string | undefined }
| { kind: "ip"; version?: IpVersion | undefined; message?: string | undefined }
| { kind: "cidr"; version?: IpVersion | undefined; message?: string | undefined }
| { kind: "base64"; message?: string | undefined }
| { kind: "base64url"; message?: string | undefined };

export interface ZodStringDef extends ZodTypeDef {
  checks: ZodStringCheck[];
  typeName: ZodFirstPartyTypeKind.ZodString;
  coerce: boolean;
}

const cuidRegex = /^c[^\s-]{8}$/i;
const cuid2Regex = /^[0-9a-z]+$/;
const ulidRegex = /^[0-9A-HJKMNP-TV-Z]{26}$/i;
// const uuidRegex =
//   /^[a-f0-9]{8}-[a-f0-9]{4}-[1-5][a-f0-9]{3}-[a-f0-9]{4}-[a-f0-9]{12}|00000000-0000-0000-0000-000000000000$/i;
const uuidRegex = /^[0-9a-fA-F]{8}\b-[0-9a-fA-F]{4}\b-[0-9a-fA-F]{4}\b-[0-9a-fA-F]{12}$/i;
const nanoidRegex = /^[a-z0-9_-]{21}$/i;
const jwtRegex = /^[A-Za-z0-9-_.]+\.[A-Za-z0-9-_.]+\.[A-Za-z0-9-_.]*$/;
const durationRegex =
  /^[-+]?P(?:?!$)(?:(?:[-+]?d+Y)|(?:[-+]?d+[.,]d+Y$))?(?:(?:[-+]?d+M)|(?:[-+]?d+[.,]d+M$))?(?:(?:[-+]?d+W)|(?:[-+]?d+[.,]d+W$))?(?:(?:[-+]?d+D)|(?:[-+]?d+[.,]d+D$))?(?:T(?:=[d+-])(?:?:[-+]?d+H)|(?:[-+]?d+[.,]d+H$))?(?:?:[-+]?d+M)|(?:[-+]?d+[.,]d+M$))?(?:[-+]?d+(?:[.,]d+)?S)?(?:?)/;

```

[illegible]

```

/^((([0-9a-fA-F]{1,4}){7,7}([0-9a-fA-F]{1,4}){1,7}|([0-9a-fA-F]{1,4}){1,6}:([0-9a-fA-F]{1,4}){1,4}|([0-9a-fA-F]{1,4}){1,5}:([0-9a-fA-F]{1,4}){1,2}|([0-9a-fA-F]{1,4}){1,4}:([0-9a-fA-F]{1,4}){1,3}|([0-9a-fA-F]{1,4}){1,4}:([0-9a-fA-F]{1,4}){1,2}:([0-9a-fA-F]{1,4}){1,5}|([0-9a-fA-F]{1,4}):([0-9a-fA-F]{1,4}){1,6}):([0-9a-fA-F]{1,4}){1,7}|:))fe80:([0-9a-fA-F]{0,4}){0,4}%[0-9a-zA-Z]{1,}:(((ffff:0{1,4}){0,1}:)|(25[0-5])(2[0-4]|1{0,1}[0-9])|0{0,1}[0-9])|((25[0-5])(2[0-4]|1{0,1}[0-9])|0{0,1}[0-9])|((25[0-5])(2[0-4]|1{0,1}[0-9])|0{0,1}[0-9]))v(12[0-8]|1[01][0-9]|1[0-9]?[0-9])$);

// https://stackoverflow.com/questions/7860392/determine-if-string-is-in-base64-using-javascript
const base64Regex = /^[0-9a-zA-Z+/]{4}*(([0-9a-zA-Z+/]{2}==)|([0-9a-zA-Z+/]{3}=))?$;/;

// https://base64.guru/standards/base64url
const base64urlRegex = /^[0-9a-zA-Z-_]{4}*(([0-9a-zA-Z-_]{2}(==)?)|([0-9a-zA-Z-_]{3}(=)?))?$;/;

// simple
// const dateRegexSource = '\\d{4}-\\d{2}-\\d{2}';
// no leap year validation
// const dateRegexSource = '\\d{4}-((0[13578]|10|12)-31|(0[13-9]|1[0-2])-30|(0[1-9]|1[0-2])-(0[1-9]|1\\d|2\\d))';
// with leap year validation
const dateRegexSource = `((\\d{4}[048][\\d{2}[13579][26]\\d{2}[048][02468][048]00|[13579][26]00)-02-29|\\d{4}-((0[13578]|1[02])-(0[1-9]|12|\\d{3}[01])|(0[469]|11)-(0[1-9]|12|\\d{3}0)|(02)-(0[1-9]|1\\d|2[0-8])))`);
const dateRegex = new RegExp(`^${dateRegexSource}$`);

function timeRegexSource(args: { precision?: number | null }) {
  let secondsRegexSource = `[0-5]\\d`;
  if (args.precision) {
    secondsRegexSource = `${secondsRegexSource}\\.\\d{${args.precision}}`;
  } else if (args.precision == null) {
    secondsRegexSource = `${secondsRegexSource}(\\.\\d+)?`;
  }
}

const secondsQuantifier = args.precision ? "+" : "?"; // require seconds if precision is nonzero
return `([01]\\d|2[0-3]):[0-5]\\d{2}(:${secondsRegexSource})${secondsQuantifier}`;
}

function timeRegex(args: {
  offset?: boolean;
  local?: boolean;
  precision?: number | null;
}) {
  return new RegExp(`^${timeRegexSource(args)}$`);
}

// Adapted from https://stackoverflow.com/a/3143231
export function datetimeRegex(args: {
  precision?: number | null;
  offset?: boolean;
  local?: boolean;
}) {
  let regex = `${dateRegexSource}T${timeRegexSource(args)}`;

  const opts: string[] = [];
  opts.push(args.local ? `Z` : `Z`);
  if (args.offset) opts.push(`([+-]\\d{2}:?\\d{2})`);
  regex = `${regex}${opts.join("")}`;
  return new RegExp(`^${regex}$`);
}

function isValidIP(ip: string, version?: IpVersion) {

```

```

    if ((version === "v4" || !version) && ipv4Regex.test(ip)) {
      return true;
    }
    if ((version === "v6" || !version) && ipv6Regex.test(ip)) {
      return true;
    }

    return false;
  }

function isValidJWT(jwt: string, alg?: string): boolean {
  if (!jwtRegex.test(jwt)) return false;
  try {
    const [header] = jwt.split(".");
    if (!header) return false;
    // Convert base64url to base64
    const base64 = header
      .replace(/-/g, "+")
      .replace(/_/g, "/")
      .padEnd(header.length + ((4 - (header.length % 4)) % 4), "=");
    // @ts-ignore
    const decoded = JSON.parse(atob(base64));
    if (typeof decoded !== "object" || decoded === null) return false;
    if ("typ" in decoded && decoded?.typ !== "JWT") return false;
    if (!decoded.alg) return false;
    if (alg && decoded.alg !== alg) return false;
    return true;
  } catch {
    return false;
  }
}

function isValidCidr(ip: string, version?: IpVersion) {
  if ((version === "v4" || !version) && ipv4CidrRegex.test(ip)) {
    return true;
  }
  if ((version === "v6" || !version) && ipv6CidrRegex.test(ip)) {
    return true;
  }

  return false;
}

export class ZodString extends ZodType<string, ZodStringDef, string> {
  _parse(input: ParseInput): ParseReturnType<string> {
    if (this._def.coerce) {
      input.data = String(input.data);
    }
    const parsedType = this._getType(input);

    if (parsedType !== ZodParsedType.string) {
      const ctx = this._getOrReturnCtx(input);
      addIssueToContext(ctx, {
        code: ZodIssueCode.invalid_type,
        expected: ZodParsedType.string,
        received: ctx.parsedType,
      });
      return INVALID;
    }
  }
}

```

```

    }

    const status = new ParseStatus();
    let ctx: undefined | ParseContext = undefined;

    for (const check of this._def.checks) {
      if (check.kind === "min") {
        if (input.data.length < check.value) {
          ctx = this._getOrReturnCtx(input, ctx);
          addIssueToContext(ctx, {
            code: ZodIssueCode.too_small,
            minimum: check.value,
            type: "string",
            inclusive: true,
            exact: false,
            message: check.message,
          });
          status.dirty();
        }
      } else if (check.kind === "max") {
        if (input.data.length > check.value) {
          ctx = this._getOrReturnCtx(input, ctx);
          addIssueToContext(ctx, {
            code: ZodIssueCode.too_big,
            maximum: check.value,
            type: "string",
            inclusive: true,
            exact: false,
            message: check.message,
          });
          status.dirty();
        }
      } else if (check.kind === "length") {
        const tooBig = input.data.length > check.value;
        const tooSmall = input.data.length < check.value;
        if (tooBig || tooSmall) {
          ctx = this._getOrReturnCtx(input, ctx);
          if (tooBig) {
            addIssueToContext(ctx, {
              code: ZodIssueCode.too_big,
              maximum: check.value,
              type: "string",
              inclusive: true,
              exact: true,
              message: check.message,
            });
          } else if (tooSmall) {
            Rest extends ZodTypeAny | null = null,
            > extends ZodType<OutputTypeOfTupleWithRest<T, Rest>, ZodTupleDef<T, Rest>, InputTypeOfTupleWithRest<T, Rest>> {
            _parse(input: ParseInput): ParseReturnType<this["_output"]> {
              const { status, ctx } = this._processInputParams(input);
              if (ctx.parsedType !== ZodParsedType.array) {
                addIssueToContext(ctx, {
                  code: ZodIssueCode.invalid_type,
                  expected: ZodParsedType.array,
                  received: ctx.parsedType,
                });
              }
            }
          }
        }
      }
    }
  }

```

```

    return INVALID;
  }

  if (ctx.data.length < this._def.items.length) {
    addIssueToContext(ctx, {
      code: ZodIssueCode.too_small,
      minimum: this._def.items.length,
      inclusive: true,
      exact: false,
      type: "array",
    });

    return INVALID;
  }

  const rest = this._def.rest;

  if (!rest && ctx.data.length > this._def.items.length) {
    addIssueToContext(ctx, {
      code: ZodIssueCode.too_big,
      maximum: this._def.items.length,
      inclusive: true,
      exact: false,
      type: "array",
    });
    status.dirty();
  }

  const items = ([...ctx.data] as any[])
    .map((item, itemIndex) => {
      const schema = this._def.items[itemIndex] || this._def.rest;
      if (!schema) return null as any as SyncParseReturnType<any>;
      return schema._parse(new ParseInputLazyPath(ctx, item, ctx.path, itemIndex));
    })
    .filter((x) => !!x); // filter nulls

  if (ctx.common.async) {
    return Promise.all(items).then((results) => {
      return ParseStatus.mergeArray(status, results);
    });
  } else {
    return ParseStatus.mergeArray(status, items as SyncParseReturnType[]);
  }
}

get items() {
  return this._def.items;
}

rest<RestSchema extends ZodTypeAny>(rest: RestSchema): ZodTuple<T, RestSchema> {
  return new ZodTuple({
    ...this._def,
    rest,
  });
}

static create = <Items extends [ZodTypeAny, ...ZodTypeAny[]] | []>(
  schemas: Items,

```

```

    params?: RawCreateParams
  ): ZodTuple<Items, null> => {
    if (!Array.isArray(schemas)) {
      throw new Error("You must pass an array of schemas to z.tuple([ ... ])");
    }
    return new ZodTuple({
      items: schemas,
      typeName: ZodFirstPartyTypeKind.ZodTuple,
      rest: null,
      ...processCreateParams(params),
    });
  };
}

////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
export interface ZodRecordDef<Key extends KeySchema = ZodString, Value extends ZodTypeAny = ZodTypeAny>
  extends ZodTypeDef {
  valueType: Value;
  keyType: Key;
  typeName: ZodFirstPartyTypeKind.ZodRecord;
}

export type KeySchema = ZodType<string | number | symbol, any, any>;
export type RecordType<K extends string | number | symbol, V> = [string] extends [K]
  ? Record<K, V>
  : [number] extends [K]
  ? Record<K, V>
  : [symbol] extends [K]
  ? Record<K, V>
  : [BRAND<string | number | symbol>] extends [K]
  ? Record<K, V>
  : Partial<Record<K, V>>;
export class ZodRecord<Key extends KeySchema = ZodString, Value extends ZodTypeAny = ZodTypeAny> extends
  ZodType<
    RecordType<Key["_output"], Value["_output"]>,
    ZodRecordDef<Key, Value>,
    RecordType<Key["_input"], Value["_input"]>
  > {
  get keySchema() {
    return this._def.keyType;
  }
  get valueSchema() {
    return this._def.valueType;
  }
  _parse(input: ParseInput): ParseReturnType<this["_output"]> {
    const { status, ctx } = this._processInputParams(input);
    if (ctx.parsedType !== ZodParsedType.object) {
      addIssueToContext(ctx, {
        code: ZodIssueCode.invalid_type,
        expected: ZodParsedType.object,
        received: ctx.parsedType,
      });
    }
  }
}

```

```

    return INVALID;
  }

  const pairs: {
    key: ParseReturnType<any>;
    value: ParseReturnType<any>;
    alwaysSet: boolean;
  }[] = [];

  const keyType = this._def.keyType;
  const valueType = this._def.valueType;

  for (const key in ctx.data) {
    pairs.push({
      key: keyType._parse(new ParseInputLazyPath(ctx, key, ctx.path, key)),
      value: valueType._parse(new ParseInputLazyPath(ctx, ctx.data[key], ctx.path, key)),
      alwaysSet: key in ctx.data,
    });
  }

  if (ctx.common.async) {
    return ParseStatus.mergeObjectAsync(status, pairs);
  } else {
    return ParseStatus.mergeObjectSync(status, pairs as any);
  }
}

get element() {
  return this._def.valueType;
}

static create<Value extends ZodTypeAny>(valueType: Value, params?: RawCreateParams): ZodRecord<ZodString, Value>;
static create<Keys extends KeySchema, Value extends ZodTypeAny>(
  keySchema: Keys,
  valueType: Value,
  params?: RawCreateParams
): ZodRecord<Keys, Value>;
static create(first: any, second?: any, third?: any): ZodRecord<any, any> {
  if (second instanceof ZodType) {
    return new ZodRecord({
      keyType: first,
      valueType: second,
      typeName: ZodFirstPartyTypeKind.ZodRecord,
      ...processCreateParams(third),
    });
  }

  return new ZodRecord({
    keyType: ZodString.create(),
    valueType: first,
    typeName: ZodFirstPartyTypeKind.ZodRecord,
    ...processCreateParams(second),
  });
}
}

////////////////////////////////////

```



```

////////////////////////////////////
//////////          //////////
//////////  ZodMap  //////////
//////////          //////////
////////////////////////////////////
////////////////////////////////////
export interface ZodMapDef<Key extends ZodTypeAny = ZodTypeAny, Value extends ZodTypeAny = ZodTypeAny>
  extends ZodTypeDef {
  valueType: Value;
  keyType: Key;
  typeName: ZodFirstPartyTypeKind.ZodMap;
}

export class ZodMap<Key extends ZodTypeAny = ZodTypeAny, Value extends ZodTypeAny = ZodTypeAny> extends
  ZodType<
    Map<Key["_output"], Value["_output"]>,
    ZodMapDef<Key, Value>,
    Map<Key["_input"], Value["_input"]>
  > {
  get keySchema() {
    return this._def.keyType;
  }
  get valueSchema() {
    return this._def.valueType;
  }
  _parse(input: ParseInput): ParseReturnType<this["_output"]> {
    const { status, ctx } = this._processInputParams(input);
    if (ctx.parsedType !== ZodParsedType.map) {
      addIssueToContext(ctx, {
        code: ZodIssueCode.invalid_type,
        expected: ZodParsedType.map,
        received: ctx.parsedType,
      });
      return INVALID;
    }

    const keyType = this._def.keyType;
    const valueType = this._def.valueType;

    const pairs = [...(ctx.data as Map<unknown, unknown>).entries()].map(([key, value], index) => {
      return {
        key: keyType._parse(new ParseInputLazyPath(ctx, key, ctx.path, [index, "key"])),
        value: valueType._parse(new ParseInputLazyPath(ctx, value, ctx.path, [index, "value"])),
      };
    });

    if (ctx.common.async) {
      const finalMap = new Map();
      return Promise.resolve().then(async () => {
        for (const pair of pairs) {
          const key = await pair.key;
          const value = await pair.value;
          if (key.status === "aborted" || value.status === "aborted") {
            return INVALID;
          }
        }
        if (key.status === "dirty" || value.status === "dirty") {
          status.dirty();
        }
      });
    }
  }
}

```

```

        finalMap.set(key.value, value.value);
    }
    return { status: status.value, value: finalMap };
});
} else {
    const finalMap = new Map();
    for (const pair of pairs) {
        const key = pair.key as SyncParseReturnType;
        const value = pair.value as SyncParseReturnType;
        if (key.status === "aborted" || value.status === "aborted") {
            return INVALID;
        }
        if (key.status === "dirty" || value.status === "dirty") {
            status.dirty();
        }

        finalMap.set(key.value, value.value);
    }
    return { status: status.value, value: finalMap };
}
}
static create = <KeySchema extends ZodTypeAny = ZodTypeAny, ValueSchema extends ZodTypeAny =
ZodTypeAny>(
    keyType: KeySchema,
    valueType: ValueSchema,
    params?: RawCreateParams
): ZodMap<KeySchema, ValueSchema> => {
    return new ZodMap({
        valueType,
        keyType,
        typeName: ZodFirstPartyTypeKind.ZodMap,
        ...processCreateParams(params),
    });
};
}

////////////////////////////////////
////////////////////////////////////
//////////      //////////
//////////  ZodSet  //////////
//////////      //////////
////////////////////////////////////
////////////////////////////////////
export interface ZodSetDef<Value extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
    valueType: Value;
    typeName: ZodFirstPartyTypeKind.ZodSet;
    minSize: { value: number; message?: string | undefined } | null;
    maxSize: { value: number; message?: string | undefined } | null;
}

export class ZodSet<Value extends ZodTypeAny = ZodTypeAny> extends ZodType<
    Set<Value["_output"]>,
    ZodSetDef<Value>,
    Set<Value["_input"]>
> {
    _parse(input: ParseInput): ParseReturnType<this["_output"]> {
        const { status, ctx } = this._processInputParams(input);

```

```

if (ctx.parsedType !== ZodParsedType.set) {
  addIssueToContext(ctx, {
    code: ZodIssueCode.invalid_type,
    expected: ZodParsedType.set,
    received: ctx.parsedType,
  });
  return INVALID;
}

const def = this._def;

if (def.minSize !== null) {
  if (ctx.data.size < def.minSize.value) {
    addIssueToContext(ctx, {
      code: ZodIssueCode.too_small,
      minimum: def.minSize.value,
      type: "set",
      inclusive: true,
      exact: false,
      message: def.minSize.message,
    });
    status.dirty();
  }
}

if (def.maxSize !== null) {
  if (ctx.data.size > def.maxSize.value) {
    addIssueToContext(ctx, {
      code: ZodIssueCode.too_big,
      maximum: def.maxSize.value,
      type: "set",
      inclusive: true,
      exact: false,
      message: def.maxSize.message,
    });
    status.dirty();
  }
}

const valueType = this._def.valueType;

function finalizeSet(elements: SyncParseReturnType<any>[]) {
  const parsedSet = new Set();
  for (const element of elements) {
    if (element.status === "aborted") return INVALID;
    if (element.status === "dirty") status.dirty();
    parsedSet.add(element.value);
  }
  return { status: status.value, value: parsedSet };
}

const elements = [...(ctx.data as Set<unknown>).values()].map((item, i) =>
  valueType._parse(new ParseInputLazyPath(ctx, item, ctx.path, i))
);

if (ctx.common.async) {
  return Promise.all(elements).then((elements) => finalizeSet(elements));
} else {

```

```

    return finalizeSet(elements as SyncParseReturnType[]);
  }
}

min(minSize: number, message?: errorUtil.ErrMessage): this {
  return new ZodSet({
    ...this._def,
    minSize: { value: minSize, message: errorUtil.toString(message) },
  }) as any;
}

max(maxSize: number, message?: errorUtil.ErrMessage): this {
  return new ZodSet({
    ...this._def,
    maxSize: { value: maxSize, message: errorUtil.toString(message) },
  }) as any;
}

size(size: number, message?: errorUtil.ErrMessage): this {
  return this.min(size, message).max(size, message) as any;
}

nonempty(message?: errorUtil.ErrMessage): ZodSet<Value> {
  return this.min(1, message) as any;
}

static create = <ValueSchema extends ZodTypeAny = ZodTypeAny>(
  valueType: ValueSchema,
  params?: RawCreateParams
): ZodSet<ValueSchema> => {
  return new ZodSet({
    valueType,
    minSize: null,
    maxSize: null,
    typeName: ZodFirstPartyTypeKind.ZodSet,
    ...processCreateParams(params),
  });
};
}

////////////////////////////////////
////////////////////////////////////
//////////      //////////
//////////  ZodFunction  //////////
//////////      //////////
////////////////////////////////////
////////////////////////////////////
export interface ZodFunctionDef<
  Args extends ZodTuple<any, any> = ZodTuple<any, any>,
  Returns extends ZodTypeAny = ZodTypeAny,
> extends ZodTypeDef {
  args: Args;
  returns: Returns;
  typeName: ZodFirstPartyTypeKind.ZodFunction;
}

export type OuterTypeOfFunction<
  Args extends ZodTuple<any, any>,

```

```

Returns extends ZodTypeAny,
> = Args["_input"] extends Array<any> ? (...args: Args["_input"]) => Returns["_output"] : never;

export type InnerTypeOfFunction<
  Args extends ZodTuple<any, any>,
  Returns extends ZodTypeAny,
> = Args["_output"] extends Array<any> ? (...args: Args["_output"]) => Returns["_input"] : never;

export class ZodFunction<Args extends ZodTuple<any, any>, Returns extends ZodTypeAny> extends ZodType<
  OuterTypeOfFunction<Args, Returns>,
  ZodFunctionDef<Args, Returns>,
  InnerTypeOfFunction<Args, Returns>
> {
  _parse(input: ParseInput): ParseReturnType<any> {
    const { ctx } = this._processInputParams(input);
    if (ctx.parsedType !== ZodParsedType.function) {
      addIssueToContext(ctx, {
        code: ZodIssueCode.invalid_type,
        expected: ZodParsedType.function,
        received: ctx.parsedType,
      });
      return INVALID;
    }
  }

  function makeArgsIssue(args: any, error: ZodError): ZodIssue {
    return makeIssue({
      data: args,
      path: ctx.path,
      errorMaps: [ctx.common.contextualErrorMap, ctx.schemaErrorMap, getErrorMap(), defaultErrorMap].filter(
        (x) => !!x
      ),
      issueData: {
        code: ZodIssueCode.invalid_arguments,
        argumentsError: error,
      },
    });
  }

  function makeReturnsIssue(returns: any, error: ZodError): ZodIssue {
    return makeIssue({
      data: returns,
      path: ctx.path,
      errorMaps: [ctx.common.contextualErrorMap, ctx.schemaErrorMap, getErrorMap(), defaultErrorMap].filter(
        (x) => !!x
      ),
      issueData: {
        code: ZodIssueCode.invalid_return_type,
        returnTypeError: error,
      },
    });
  }

  const params = { errorMap: ctx.common.contextualErrorMap };
  const fn = ctx.data;

  if (this._def.returns instanceof ZodPromise) {
    // Would love a way to avoid disabling this rule, but we need
    // an alias (using an arrow function was what caused 2651).

```

```

// eslint-disable-next-line @typescript-eslint/no-this-alias
const me = this;
return OK(async function (this: any, ...args: any[]) {
  const error = new ZodError([]);
  const parsedArgs = await me._def.args.parseAsync(args, params).catch((e) => {
    error.addIssue(makeArgsIssue(args, e));
    throw error;
  });
  const result = await Reflect.apply(fn, this, parsedArgs as any);
  const parsedReturns = await (me._def.returns as unknown as ZodPromise<ZodTypeAny>)._def.type
    .parseAsync(result, params)
    .catch((e) => {
      error.addIssue(makeReturnsIssue(result, e));
      throw error;
    });
  return parsedReturns;
});
} else {
  // Would love a way to avoid disabling this rule, but we need
  // an alias (using an arrow function was what caused 2651).
  // eslint-disable-next-line @typescript-eslint/no-this-alias
  const me = this;
  return OK(function (this: any, ...args: any[]) {
    const parsedArgs = me._def.args.safeParse(args, params);
    if (!parsedArgs.success) {
      throw new ZodError([makeArgsIssue(args, parsedArgs.error)]);
    }
    const result = Reflect.apply(fn, this, parsedArgs.data);
    const parsedReturns = me._def.returns.safeParse(result, params);
    if (!parsedReturns.success) {
      throw new ZodError([makeReturnsIssue(result, parsedReturns.error)]);
    }
    return parsedReturns.data;
  }) as any;
}
}

parameters() {
  return this._def.args;
}

returnType() {
  return this._def.returns;
}

args<Items extends Parameters<(typeof ZodTuple)["create"]>[0]>(
  ...items: Items
): ZodFunction<ZodTuple<Items, ZodUnknown>, Returns> {
  return new ZodFunction({
    ...this._def,
    args: ZodTuple.create(items).rest(ZodUnknown.create()) as any,
  });
}

returns<NewReturnType extends ZodType<any, any, any>>(returnType: NewReturnType): ZodFunction<Args,
NewReturnType> {
  return new ZodFunction({
    ...this._def,

```

```

    returns: returnType,
  });
}

implement<F extends InnerTypeOfFunction<Args, Returns>>(
  func: F
): Return<F> extends Returns["_output"]
? (...args: Args["_input"]) => Return<F>
: OuterTypeOfFunction<Args, Returns> {
  const validatedFunc = this.parse(func);
  return validatedFunc as any;
}

strictImplement(func: InnerTypeOfFunction<Args, Returns>): InnerTypeOfFunction<Args, Returns> {
  const validatedFunc = this.parse(func);
  return validatedFunc as any;
}

validate = this.implement;

static create(): ZodFunction<ZodTuple<[], ZodUnknown>, ZodUnknown>;
static create<T extends AnyZodTuple = ZodTuple<[], ZodUnknown>>(args: T): ZodFunction<T, ZodUnknown>;
static create<T extends AnyZodTuple, U extends ZodTypeAny>(args: T, returns: U): ZodFunction<T, U>;
static create<T extends AnyZodTuple = ZodTuple<[], ZodUnknown>, U extends ZodTypeAny = ZodUnknown>(
  args: T,
  returns: U,
  params?: RawCreateParams
): ZodFunction<T, U>;
static create(args?: AnyZodTuple, returns?: ZodTypeAny, params?: RawCreateParams) {
  return new ZodFunction({
    args: (args ? args : ZodTuple.create([])).rest(ZodUnknown.create()),
    returns: returns || ZodUnknown.create(),
    typeName: ZodFirstPartyTypeKind.ZodFunction,
    ...processCreateParams(params),
  }) as any;
}

////////////////////////////////////////
////////////////////////////////////////
//////////      //////////
//////////   ZodLazy   //////////
//////////      //////////
////////////////////////////////////////
////////////////////////////////////////

export interface ZodLazyDef<T extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
  getter: () => T;
  typeName: ZodFirstPartyTypeKind.ZodLazy;
}

export class ZodLazy<T extends ZodTypeAny> extends ZodType<output<T>, ZodLazyDef<T>, input<T>> {
  get schema(): T {
    return this._def.getter();
  }

  _parse(input: ParseInput): ParseReturn<this["_output"]> {
    const { ctx } = this._processInputParams(input);
    const lazySchema = this._def.getter();

```

[illegible]


```

export type ArrayKeys = keyof any[];
export type Indices<T> = Exclude<keyof T, ArrayKeys>;

export type EnumValues<T extends string = string> = readonly [T, ...T[]];

export type Values<T extends EnumValues> = {
  [k in T[number]]: k;
};

export interface ZodEnumDef<T extends EnumValues = EnumValues> extends ZodTypeDef {
  values: T;
  typeName: ZodFirstPartyTypeKind.ZodEnum;
}

export type Writeable<T> = { -readonly [P in keyof T]: T[P] };

export type FilterEnum<Values, ToExclude> = Values extends []
  ? []
  : Values extends [infer Head, ...infer Rest]
    ? Head extends ToExclude
      ? FilterEnum<Rest, ToExclude>
      : [Head, ...FilterEnum<Rest, ToExclude>]
    : never;

export type typecast<A, T> = A extends T ? A : never;

function createZodEnum<U extends string, T extends Readonly<[U, ...U[]]>>>(
  values: T,
  params?: RawCreateParams
): ZodEnum<Writeable<T>>;
function createZodEnum<U extends string, T extends [U, ...U[]]>(values: T, params?: RawCreateParams):
  ZodEnum<T>;
function createZodEnum(values: [string, ...string[]], params?: RawCreateParams) {
  return new ZodEnum({
    values,
    typeName: ZodFirstPartyTypeKind.ZodEnum,
    ...processCreateParams(params),
  });
}

export class ZodEnum<T extends [string, ...string[]]> extends ZodType<T[number], ZodEnumDef<T>, T[number]> {
  _cache: Set<T[number]> | undefined;

  _parse(input: ParseInput): ParseReturnType<this["_output"]> {
    if (typeof input.data !== "string") {
      const ctx = this._getOrReturnCtx(input);
      const expectedValues = this._def.values;
      addIssueToContext(ctx, {
        expected: util.joinValues(expectedValues) as "string",
        received: ctx.parsedType,
        code: ZodIssueCode.invalid_type,
      });
      return INVALID;
    }

    if (!this._cache) {
      this._cache = new Set(this._def.values);
    }
  }
}

```

```

if (!this._cache.has(input.data)) {
  const ctx = this._getOrReturnCtx(input);
  const expectedValues = this._def.values;

  addIssueToContext(ctx, {
    received: ctx.data,
    code: ZodIssueCode.invalid_enum_value,
    options: expectedValues,
  });
  return INVALID;
}
return OK(input.data);
}

get options() {
  return this._def.values;
}

get enum(): Values<T> {
  const enumValues: any = {};
  for (const val of this._def.values) {
    enumValues[val] = val;
  }
  return enumValues;
}

get Values(): Values<T> {
  const enumValues: any = {};
  for (const val of this._def.values) {
    enumValues[val] = val;
  }
  return enumValues;
}

get Enum(): Values<T> {
  const enumValues: any = {};
  for (const val of this._def.values) {
    enumValues[val] = val;
  }
  return enumValues;
}

extract<ToExtract extends readonly [T[number], ...T[number][]]>(
  values: ToExtract,
  newDef: RawCreateParams = this._def
): ZodEnum<Writable<ToExtract>> {
  return ZodEnum.create(values, {
    ...this._def,
    ...newDef,
  }) as any;
}

exclude<ToExclude extends readonly [T[number], ...T[number][]]>(
  values: ToExclude,
  newDef: RawCreateParams = this._def
): ZodEnum<typecast<Writable<FilterEnum<T, ToExclude[number]>>, [string, ...string[]]>> {
  return ZodEnum.create(this.options.filter((opt) => !values.includes(opt)) as FilterEnum<T, ToExclude[number]>, {

```

```

        ...this._def,
        ...newDef,
    }) as any;
}

static create = createZodEnum;
}

////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
export interface ZodNativeEnumDef<T extends EnumLike = EnumLike> extends ZodTypeDef {
    values: T;
    typeName: ZodFirstPartyTypeKind.ZodNativeEnum;
}

export type EnumLike = { [k: string]: string | number; [nu: number]: string };

export class ZodNativeEnum<T extends EnumLike> extends ZodType<T[keyof T], ZodNativeEnumDef<T>, T[keyof T]>
{
    _cache: Set<T[keyof T]> | undefined;
    _parse(input: ParseInput): ParseReturnType<T[keyof T]> {
        const nativeEnumValues = util.getValidEnumValues(this._def.values);

        const ctx = this._getOrReturnCtx(input);
        if (ctx.parsedType !== ZodParsedType.string && ctx.parsedType !== ZodParsedType.number) {
            const expectedValues = util.objectValues(nativeEnumValues);
            addIssueToContext(ctx, {
                expected: util.joinValues(expectedValues) as "string",
                received: ctx.parsedType,
                code: ZodIssueCode.invalid_type,
            });
            return INVALID;
        }

        if (!this._cache) {
            this._cache = new Set(util.getValidEnumValues(this._def.values));
        }

        if (!this._cache.has(input.data)) {
            const expectedValues = util.objectValues(nativeEnumValues);

            addIssueToContext(ctx, {
                received: ctx.data,
                code: ZodIssueCode.invalid_enum_value,
                options: expectedValues,
            });
            return INVALID;
        }
        return OK(input.data);
    }

    get enum() {
        return this._def.values;
    }
}

```

```

    }

    static create = <Elements extends EnumLike>(values: Elements, params?: RawCreateParams):
    ZodNativeEnum<Elements> => {
      return new ZodNativeEnum({
        values: values,
        typeName: ZodFirstPartyTypeKind.ZodNativeEnum,
        ...processCreateParams(params),
      });
    };
  }

  ///////////////////////////////////////////////////
  ///////////////////////////////////////////////////
  ///////////////          ///////////////
  ///////////////    ZodPromise    ///////////////
  ///////////////          ///////////////
  ///////////////////////////////////////////////////
  ///////////////////////////////////////////////////
  export interface ZodPromiseDef<T extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
    type: T;
    typeName: ZodFirstPartyTypeKind.ZodPromise;
  }

  export class ZodPromise<T extends ZodTypeAny> extends ZodType<
    Promise<T["_output"]>,
    ZodPromiseDef<T>,
    Promise<T["_input"]>
  > {
    unwrap() {
      return this._def.type;
    }

    _parse(input: ParseInput): ParseReturnType<this["_output"]> {
      const { ctx } = this._processInputParams(input);
      if (ctx.parsedType !== ZodParsedType.promise && ctx.common.async === false) {
        addIssueToContext(ctx, {
          code: ZodIssueCode.invalid_type,
          expected: ZodParsedType.promise,
          received: ctx.parsedType,
        });
        return INVALID;
      }

      const promisified = ctx.parsedType === ZodParsedType.promise ? ctx.data : Promise.resolve(ctx.data);

      return OK(
        promisified.then((data: any) => {
          return this._def.type.parseAsync(data, {
            path: ctx.path,
            errorMap: ctx.common.contextualErrorMap,
          });
        })
      );
    }
  }

  static create = <Inner extends ZodTypeAny>(schema: Inner, params?: RawCreateParams): ZodPromise<Inner> => {
    return new ZodPromise({

```

```

    type: schema,
    typeName: ZodFirstPartyTypeKind.ZodPromise,
    ...processCreateParams(params),
  });
};
}

////////////////////////////////////
////////////////////////////////////
//////////      //////////
//////////      ZodEffects      //////////
//////////      //////////
////////////////////////////////////
////////////////////////////////////

export type Refinement<T> = (arg: T, ctx: RefinementCtx) => any;
export type SuperRefinement<T> = (arg: T, ctx: RefinementCtx) => void | Promise<void>;

export type RefinementEffect<T> = {
  type: "refinement";
  refinement: (arg: T, ctx: RefinementCtx) => any;
};
export type TransformEffect<T> = {
  type: "transform";
  transform: (arg: T, ctx: RefinementCtx) => any;
};
export type PreprocessEffect<T> = {
  type: "preprocess";
  transform: (arg: T, ctx: RefinementCtx) => any;
};
export type Effect<T> = RefinementEffect<T> | TransformEffect<T> | PreprocessEffect<T>;

export interface ZodEffectsDef<T> extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
  schema: T;
  typeName: ZodFirstPartyTypeKind.ZodEffects;
  effect: Effect<any>;
}

export class ZodEffects<T> extends ZodTypeAny, Output = output<T>, Input = input<T>> extends ZodType<
  Output,
  ZodEffectsDef<T>,
  Input
> {
  innerType() {
    return this._def.schema;
  }

  sourceType(): T {
    return this._def.schema._def.typeName === ZodFirstPartyTypeKind.ZodEffects
      ? (this._def.schema as unknown as ZodEffects<T>).sourceType()
      : (this._def.schema as T);
  }

  _parse(input: ParseInput): ParseReturnType<this["_output"]> {
    const { status, ctx } = this._processInputParams(input);

    const effect = this._def.effect || null;

```

```

const checkCtx: RefinementCtx = {
  addIssue: (arg: IssueData) => {
    addIssueToContext(ctx, arg);
    if (arg.fatal) {
      status.abort();
    } else {
      status.dirty();
    }
  },
  get path() {
    return ctx.path;
  },
};

checkCtx.addIssue = checkCtx.addIssue.bind(checkCtx);

if (effect.type === "preprocess") {
  const processed = effect.transform(ctx.data, checkCtx);

  if (ctx.common.async) {
    return Promise.resolve(processed).then(async (processed) => {
      if (status.value === "aborted") return INVALID;

      const result = await this._def.schema._parseAsync({
        data: processed,
        path: ctx.path,
        parent: ctx,
      });
      if (result.status === "aborted") return INVALID;
      if (result.status === "dirty") return DIRTY(result.value);
      if (status.value === "dirty") return DIRTY(result.value);
      return result;
    });
  } else {
    if (status.value === "aborted") return INVALID;
    const result = this._def.schema._parseSync({
      data: processed,
      path: ctx.path,
      parent: ctx,
    });
    if (result.status === "aborted") return INVALID;
    if (result.status === "dirty") return DIRTY(result.value);
    if (status.value === "dirty") return DIRTY(result.value);
    return result;
  }
}

if (effect.type === "refinement") {
  const executeRefinement = (acc: unknown): any => {
    const result = effect.refinement(acc, checkCtx);
    if (ctx.common.async) {
      return Promise.resolve(result);
    }
    if (result instanceof Promise) {
      throw new Error("Async refinement encountered during synchronous parse operation. Use .parseAsync instead.");
    }
    return acc;
  };
};

```

```

if (ctx.common.async === false) {
  const inner = this._def.schema._parseSync({
    data: ctx.data,
    path: ctx.path,
    parent: ctx,
  });
  if (inner.status === "aborted") return INVALID;
  if (inner.status === "dirty") status.dirty();

  // return value is ignored
  executeRefinement(inner.value);
  return { status: status.value, value: inner.value };
} else {
  return this._def.schema._parseAsync({ data: ctx.data, path: ctx.path, parent: ctx }).then((inner) => {
    if (inner.status === "aborted") return INVALID;
    if (inner.status === "dirty") status.dirty();

    return executeRefinement(inner.value).then(() => {
      return { status: status.value, value: inner.value };
    });
  });
}

if (effect.type === "transform") {
  if (ctx.common.async === false) {
    const base = this._def.schema._parseSync({
      data: ctx.data,
      path: ctx.path,
      parent: ctx,
    });

    if (!isValid(base)) return INVALID;

    const result = effect.transform(base.value, checkCtx);
    if (result instanceof Promise) {
      throw new Error(
        `Asynchronous transform encountered during synchronous parse operation. Use .parseAsync instead.`
      );
    }

    return { status: status.value, value: result };
  } else {
    return this._def.schema._parseAsync({ data: ctx.data, path: ctx.path, parent: ctx }).then((base) => {
      if (!isValid(base)) return INVALID;

      return Promise.resolve(effect.transform(base.value, checkCtx)).then((result) => ({
        status: status.value,
        value: result,
      }));
    });
  }
}

util.assertNever(effect);
}

```

```

static create = <I extends ZodTypeAny>(
  schema: I,
  effect: Effect<I["_output"]>,
  params?: RawCreateParams
): ZodEffects<I, I["_output"]> => {
  return new ZodEffects({
    schema,
    typeName: ZodFirstPartyTypeKind.ZodEffects,
    effect,
    ...processCreateParams(params),
  });
};

static createWithPreprocess = <I extends ZodTypeAny>(
  preprocess: (arg: unknown, ctx: RefinementCtx) => unknown,
  schema: I,
  params?: RawCreateParams
): ZodEffects<I, I["_output"], unknown> => {
  return new ZodEffects({
    schema,
    effect: { type: "preprocess", transform: preprocess },
    typeName: ZodFirstPartyTypeKind.ZodEffects,
    ...processCreateParams(params),
  });
};
}

export { ZodEffects as ZodTransformer };

////////////////////////////////////
////////////////////////////////////
/////////      ///////////
/////////  ZodOptional  ///////////
/////////      ///////////
////////////////////////////////////
////////////////////////////////////
export interface ZodOptionalDef<T extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
  innerType: T;
  typeName: ZodFirstPartyTypeKind.ZodOptional;
}

export type ZodOptionalType<T extends ZodTypeAny> = ZodOptional<T>;

export class ZodOptional<T extends ZodTypeAny> extends ZodType<
  T["_output"] | undefined,
  ZodOptionalDef<T>,
  T["_input"] | undefined
> {
  _parse(input: ParseInput): ParseReturnType<this["_output"]> {
    const parsedType = this._getType(input);
    if (parsedType === ZodParsedType.undefined) {
      return OK(undefined);
    }
    return this._def.innerType._parse(input);
  }

  unwrap() {
    return this._def.innerType;
  }
}

```



```

    }

    static create = <Inner extends ZodTypeAny>(type: Inner, params?: RawCreateParams): ZodOptional<Inner> => {
      return new ZodOptional({
        innerType: type,
        typeName: ZodFirstPartyTypeKind.ZodOptional,
        ...processCreateParams(params),
      }) as any;
    };
  }

  ///////////////////////////////////////////////////
  ///////////////////////////////////////////////////
  ///////////////          ///////////////
  ///////////////   ZodNullable   ///////////////
  ///////////////          ///////////////
  ///////////////////////////////////////////////////
  ///////////////////////////////////////////////////
  export interface ZodNullableDef<T extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
    innerType: T;
    typeName: ZodFirstPartyTypeKind.ZodNullable;
  }

  export type ZodNullableType<T extends ZodTypeAny> = ZodNullable<T>;

  export class ZodNullable<T extends ZodTypeAny> extends ZodType<
    T["_output"] | null,
    ZodNullableDef<T>,
    T["_input"] | null
  > {
    _parse(input: ParseInput): ParseReturnType<this["_output"]> {
      const parsedType = this._getType(input);
      if (parsedType === ZodParsedType.null) {
        return OK(null);
      }
      return this._def.innerType._parse(input);
    }

    unwrap() {
      return this._def.innerType;
    }

    static create = <Inner extends ZodTypeAny>(type: Inner, params?: RawCreateParams): ZodNullable<Inner> => {
      return new ZodNullable({
        innerType: type,
        typeName: ZodFirstPartyTypeKind.ZodNullable,
        ...processCreateParams(params),
      }) as any;
    };
  }

  ///////////////////////////////////////////////////
  ///////////////////////////////////////////////////
  ///////////////          ///////////////
  ///////////////   ZodDefault   ///////////////
  ///////////////          ///////////////
  ///////////////////////////////////////////////////
  ///////////////////////////////////////////////////

```

```

export interface ZodDefaultDef<T extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
  innerType: T;
  defaultValue: () => util.noUndefined<T["_input"]>;
  typeName: ZodFirstPartyTypeKind.ZodDefault;
}

export class ZodDefault<T extends ZodTypeAny> extends ZodType<
  util.noUndefined<T["_output"]>,
  ZodDefaultDef<T>,
  T["_input"] | undefined
> {
  _parse(input: ParseInput): ParseReturnType<this["_output"]> {
    const { ctx } = this._processInputParams(input);
    let data = ctx.data;
    if (ctx.parsedType === ZodParsedType.undefined) {
      data = this._def.defaultValue();
    }
    return this._def.innerType._parse({
      data,
      path: ctx.path,
      parent: ctx,
    });
  }

  removeDefault() {
    return this._def.innerType;
  }

  static create = <Inner extends ZodTypeAny>(
    type: Inner,
    params: RawCreateParams & {
      default: Inner["_input"] | (() => util.noUndefined<Inner["_input"]>);
    }
  ): ZodDefault<Inner> => {
    return new ZodDefault({
      innerType: type,
      typeName: ZodFirstPartyTypeKind.ZodDefault,
      defaultValue: typeof params.default === "function" ? params.default : () => params.default as any,
      ...processCreateParams(params),
    }) as any;
  };
}

////////////////////////////////////
////////////////////////////////////
//////////      //////////
//////////  ZodCatch  //////////
//////////      //////////
////////////////////////////////////
////////////////////////////////////
export interface ZodCatchDef<T extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
  innerType: T;
  catchValue: (ctx: { error: ZodError; input: unknown }) => T["_input"];
  typeName: ZodFirstPartyTypeKind.ZodCatch;
}

export class ZodCatch<T extends ZodTypeAny> extends ZodType<
  T["_output"],

```

```

ZodCatchDef<T>,
unknown // any input will pass validation // T["_input"]
> {
  _parse(input: ParseInput): ParseReturnType<this["_output"]> {
    const { ctx } = this._processInputParams(input);

    // newCtx is used to not collect issues from inner types in ctx
    const newCtx: ParseContext = {
      ...ctx,
      common: {
        ...ctx.common,
        issues: [],
      },
    };

    const result = this._def.innerType._parse({
      data: newCtx.data,
      path: newCtx.path,
      parent: {
        ...newCtx,
      },
    });

    if (isAsync(result)) {
      return result.then((result) => {
        return {
          status: "valid",
          value:
            result.status === "valid"
              ? result.value
              : this._def.catchValue({
                  get error() {
                    return new ZodError(newCtx.common.issues);
                  },
                  input: newCtx.data,
                }),
        };
      });
    } else {
      return {
        status: "valid",
        value:
          result.status === "valid"
            ? result.value
            : this._def.catchValue({
                get error() {
                  return new ZodError(newCtx.common.issues);
                },
                input: newCtx.data,
              }),
      };
    }
  }
}

removeCatch() {
  return this._def.innerType;
}

```

```

static create = <Inner extends ZodTypeAny>(
  type: Inner,
  params: RawCreateParams & {
    catch: Inner["_output"] | (() => Inner["_output"]);
  }
): ZodCatch<Inner> => {
  return new ZodCatch({
    innerType: type,
    typeName: ZodFirstPartyTypeKind.ZodCatch,
    catchValue: typeof params.catch === "function" ? params.catch : () => params.catch,
    ...processCreateParams(params),
  });
};
}

```

```

////////////////////////////////////
////////////////////////////////////
//////////              //////////
//////////   ZodNaN   //////////
//////////              //////////
////////////////////////////////////
////////////////////////////////////

```

```

export interface ZodNaNDef extends ZodTypeDef {
  typeName: ZodFirstPartyTypeKind.ZodNaN;
}

```

```

export class ZodNaN extends ZodType<number, ZodNaNDef, number> {
  _parse(input: ParseInput): ParseReturnType<any> {
    const parsedType = this._getType(input);
    if (parsedType !== ZodParsedType.nan) {
      const ctx = this._getOrReturnCtx(input);
      addIssueToContext(ctx, {
        code: ZodIssueCode.invalid_type,
        expected: ZodParsedType.nan,
        received: ctx.parsedType,
      });
      return INVALID;
    }
  }

  return { status: "valid", value: input.data };
}

```

```

static create = (params?: RawCreateParams): ZodNaN => {
  return new ZodNaN({
    typeName: ZodFirstPartyTypeKind.ZodNaN,
    ...processCreateParams(params),
  });
};
}

```

```

////////////////////////////////////
////////////////////////////////////
//////////              //////////
//////////   ZodBranded //////////
//////////              //////////
////////////////////////////////////
////////////////////////////////////

```

```

export interface ZodBrandedDef<T extends ZodTypeAny> extends ZodTypeDef {
  type: T;
  typeName: ZodFirstPartyTypeKind.ZodBranded;
}

export const BRAND: unique symbol = Symbol("zod_brand");
export type BRAND<T extends string | number | symbol> = {
  [BRAND]: { [k in T]: true };
};

export class ZodBranded<T extends ZodTypeAny, B extends string | number | symbol> extends ZodType<
  T["_output"] & BRAND<B>,
  ZodBrandedDef<T>,
  T["_input"]
> {
  _parse(input: ParseInput): ParseReturnType<any> {
    const { ctx } = this._processInputParams(input);
    const data = ctx.data;
    return this._def.type._parse({
      data,
      path: ctx.path,
      parent: ctx,
    });
  }

  unwrap() {
    return this._def.type;
  }
}

////////////////////////////////////
////////////////////////////////////
//////////      //////////
//////////  ZodPipeline  //////////
//////////      //////////
////////////////////////////////////
////////////////////////////////////

export interface ZodPipelineDef<A extends ZodTypeAny, B extends ZodTypeAny> extends ZodTypeDef {
  in: A;
  out: B;
  typeName: ZodFirstPartyTypeKind.ZodPipeline;
}

export class ZodPipeline<A extends ZodTypeAny, B extends ZodTypeAny> extends ZodType<
  B["_output"],
  ZodPipelineDef<A, B>,
  A["_input"]
> {
  _parse(input: ParseInput): ParseReturnType<any> {
    const { status, ctx } = this._processInputParams(input);
    if (ctx.common.async) {
      const handleAsync = async () => {
        const inResult = await this._def.in._parseAsync({
          data: ctx.data,
          path: ctx.path,
          parent: ctx,
        });
      };
    }
  }
}

```

```

    });
    if (inResult.status === "aborted") return INVALID;
    if (inResult.status === "dirty") {
      status.dirty();
      return DIRTY(inResult.value);
    } else {
      return this._def.out._parseAsync({
        data: inResult.value,
        path: ctx.path,
        parent: ctx,
      });
    }
  };
  return handleAsync();
} else {
  const inResult = this._def.in._parseSync({
    data: ctx.data,
    path: ctx.path,
    parent: ctx,
  });
  if (inResult.status === "aborted") return INVALID;
  if (inResult.status === "dirty") {
    status.dirty();
    return {
      status: "dirty",
      value: inResult.value,
    };
  } else {
    return this._def.out._parseSync({
      data: inResult.value,
      path: ctx.path,
      parent: ctx,
    });
  }
}
}
}

static create<ASchema extends ZodTypeAny, BSchema extends ZodTypeAny>(
  a: ASchema,
  b: BSchema
): ZodPipeline<ASchema, BSchema> {
  return new ZodPipeline({
    in: a,
    out: b,
    typeName: ZodFirstPartyTypeKind.ZodPipeline,
  });
}

////////////////////////////////////
////////////////////////////////////
//////////      //////////
//////////  ZodReadonly  //////////
//////////      //////////
////////////////////////////////////
////////////////////////////////////
type BuiltIn =
  | (((...args: any[]) => any) | (new (...args: any[]) => any))

```

```

| { readonly [Symbol.toStringTag]: string }
| Date
| Error
| Generator
| Promise<unknown>
| RegExp;

type MakeReadOnly<T> = T extends Map<infer K, infer V>
  ? ReadonlyMap<K, V>
  : T extends Set<infer V>
  ? ReadonlySet<V>
  : T extends [infer Head, ...infer Tail]
  ? readonly [Head, ...Tail]
  : T extends Array<infer V>
  ? ReadonlyArray<V>
  : T extends BuiltIn
  ? T
  : Readonly<T>;

export interface ZodReadOnlyDef<T extends ZodTypeAny = ZodTypeAny> extends ZodTypeDef {
  innerType: T;
  typeName: ZodFirstPartyTypeKind.ZodReadOnly;
}

export class ZodReadOnly<T extends ZodTypeAny> extends ZodType<
  MakeReadOnly<T["_output"]>,
  ZodReadOnlyDef<T>,
  MakeReadOnly<T["_input"]>
> {
  _parse(input: ParseInput): ParseReturnType<this["_output"]> {
    const result = this._def.innerType._parse(input);
    const freeze = (data: ParseReturnType<this["_output"]>) => {
      if (isValid(data)) {
        data.value = Object.freeze(data.value);
      }
    }
    return data;
  };
  return isAsync(result) ? result.then((data) => freeze(data)) : freeze(result);
}

static create = <Inner extends ZodTypeAny>(type: Inner, params?: RawCreateParams): ZodReadOnly<Inner> => {
  return new ZodReadOnly({
    innerType: type,
    typeName: ZodFirstPartyTypeKind.ZodReadOnly,
    ...processCreateParams(params),
  }) as any;
};

unwrap() {
  return this._def.innerType;
}
}

//////////
//////////
//////////      //////////
//////////    z.custom  //////////
//////////      //////////

```

```

////////////////////////////////////
////////////////////////////////////
function cleanParams(params: unknown, data: unknown) {
  const p = typeof params === "function" ? params(data) : typeof params === "string" ? { message: params } : params;

  const p2 = typeof p === "string" ? { message: p } : p;
  return p2;
}
type CustomParams = CustomErrorParams & { fatal?: boolean };
export function custom<T>(
  check?: (data: any) => any,
  _params: string | CustomParams | ((input: any) => CustomParams) = {},
  /**
   * @deprecated
   *
   * Pass `fatal` into the params object instead:
   *
   * ```ts
   * z.string().custom((val) => val.length > 5, { fatal: false })
   * ```
   */
  fatal?: boolean
): ZodType<T, ZodTypeDef, T> {
  if (check)
    return ZodAny.create().superRefine((data, ctx) => {
      const r = check(data);
      if (r instanceof Promise) {
        return r.then((r) => {
          if (!r) {
            const params = cleanParams(_params, data);
            const _fatal = params.fatal ?? fatal ?? true;
            ctx.addIssue({ code: "custom", ...params, fatal: _fatal });
          }
        });
      }
    });
  if (!r) {
    const params = cleanParams(_params, data);
    const _fatal = params.fatal ?? fatal ?? true;
    ctx.addIssue({ code: "custom", ...params, fatal: _fatal });
  }
  return;
});
return ZodAny.create();
}

export { ZodType as Schema, ZodType as ZodSchema };

export const late = {
  object: ZodObject.lazycreate,
};

export enum ZodFirstPartyTypeKind {
  ZodString = "ZodString",
  ZodNumber = "ZodNumber",
  ZodNaN = "ZodNaN",
  ZodBigInt = "ZodBigInt",
  ZodBoolean = "ZodBoolean",

```



```

ZodDate = "ZodDate",
ZodSymbol = "ZodSymbol",
ZodUndefined = "ZodUndefined",
ZodNull = "ZodNull",
ZodAny = "ZodAny",
ZodUnknown = "ZodUnknown",
ZodNever = "ZodNever",
ZodVoid = "ZodVoid",
ZodArray = "ZodArray",
ZodObject = "ZodObject",
ZodUnion = "ZodUnion",
ZodDiscriminatedUnion = "ZodDiscriminatedUnion",
ZodIntersection = "ZodIntersection",
ZodTuple = "ZodTuple",
ZodRecord = "ZodRecord",
ZodMap = "ZodMap",
ZodSet = "ZodSet",
ZodFunction = "ZodFunction",
ZodLazy = "ZodLazy",
ZodLiteral = "ZodLiteral",
ZodEnum = "ZodEnum",
ZodEffects = "ZodEffects",
ZodNativeEnum = "ZodNativeEnum",
ZodOptional = "ZodOptional",
ZodNullable = "ZodNullable",
ZodDefault = "ZodDefault",
ZodCatch = "ZodCatch",
ZodPromise = "ZodPromise",
ZodBranded = "ZodBranded",
ZodPipeline = "ZodPipeline",
ZodReadonly = "ZodReadonly",
}
export type ZodFirstPartySchemaTypes =
  | ZodString
  | ZodNumber
  | ZodNaN
  | ZodBigInt
  | ZodBoolean
  | ZodDate
  | ZodUndefined
  | ZodNull
  | ZodAny
  | ZodUnknown
  | ZodNever
  | ZodVoid
  | ZodArray<any, any>
  | ZodObject<any, any, any>
  | ZodUnion<any>
  | ZodDiscriminatedUnion<any, any>
  | ZodIntersection<any, any>
  | ZodTuple<any, any>
  | ZodRecord<any, any>
  | ZodMap<any>
  | ZodSet<any>
  | ZodFunction<any, any>
  | ZodLazy<any>
  | ZodLiteral<any>
  | ZodEnum<any>

```

```

| ZodEffects<any, any, any>
| ZodNativeEnum<any>
| ZodOptional<any>
| ZodNullable<any>
| ZodDefault<any>
| ZodCatch<any>
| ZodPromise<any>
| ZodBranded<any, any>
| ZodPipeline<any, any>
| ZodReadonly<any>
| ZodSymbol;

// requires TS 4.4+
abstract class Class {
  constructor(..._: any[]) {}
}
const instanceOfType = <T extends typeof Class>(
  // const instanceOfType = <T extends new (...args: any[]) => any>(
  cls: T,
  params: CustomParams = {
    message: `Input not instance of ${cls.name}`,
  }
) => custom<InstanceType<T>>((data) => data instanceof cls, params);

const stringType = ZodString.create;
const numberType = ZodNumber.create;
const nanType = ZodNaN.create;
const bigIntType = ZodBigInt.create;
const booleanType = ZodBoolean.create;
const dateType = ZodDate.create;
const symbolType = ZodSymbol.create;
const undefinedType = ZodUndefined.create;
const nullType = ZodNull.create;
const anyType = ZodAny.create;
const unknownType = ZodUnknown.create;
const neverType = ZodNever.create;
const voidType = ZodVoid.create;
const arrayType = ZodArray.create;
const objectType = ZodObject.create;
const strictObjectType = ZodObject.strictCreate;
const unionType = ZodUnion.create;
const discriminatedUnionType = ZodDiscriminatedUnion.create;
const intersectionType = ZodIntersection.create;
const tupleType = ZodTuple.create;
const recordType = ZodRecord.create;
const mapType = ZodMap.create;
const setType = ZodSet.create;
const functionType = ZodFunction.create;
const lazyType = ZodLazy.create;
const literalType = ZodLiteral.create;
const enumType = ZodEnum.create;
const nativeEnumType = ZodNativeEnum.create;
const promiseType = ZodPromise.create;
const effectsType = ZodEffects.create;
const optionalType = ZodOptional.create;
const nullableType = ZodNullable.create;
const preprocessType = ZodEffects.createWithPreprocess;
const pipelineType = ZodPipeline.create;

```

```

const ostring = () => stringType().optional();
const onumber = () => numberType().optional();
const oboolean = () => booleanType().optional();

export const coerce = {
  string: ((arg) => ZodString.create({ ...arg, coerce: true })) as (typeof ZodString)["create"],
  number: ((arg) => ZodNumber.create({ ...arg, coerce: true })) as (typeof ZodNumber)["create"],
  boolean: ((arg) =>
    ZodBoolean.create({
      ...arg,
      coerce: true,
    })) as (typeof ZodBoolean)["create"],
  bigint: ((arg) => ZodBigInt.create({ ...arg, coerce: true })) as (typeof ZodBigInt)["create"],
  date: ((arg) => ZodDate.create({ ...arg, coerce: true })) as (typeof ZodDate)["create"],
};

export {
  anyType as any,
  arrayType as array,
  bigintType as bigint,
  booleanType as boolean,
  dateType as date,
  discriminatedUnionType as discriminatedUnion,
  effectsType as effect,
  enumType as enum,
  functionType as function,
  instanceOfType as instanceof,
  intersectionType as intersection,
  lazyType as lazy,
  literalType as literal,
  mapType as map,
  nanType as nan,
  nativeEnumType as nativeEnum,
  neverType as never,
  nullType as null,
  nullableType as nullable,
  numberType as number,
  objectType as object,
  oboolean,
  onumber,
  optionalType as optional,
  ostring,
  pipelineType as pipeline,
  preprocessType as preprocess,
  promiseType as promise,
  recordType as record,
  setType as set,
  strictObjectType as strictObject,
  stringType as string,
  symbolType as symbol,
  effectsType as transformer,
  tupleType as tuple,
  undefinedType as undefined,
  unionType as union,
  unknownType as unknown,
  voidType as void,
};

```

```
export const NEVER = INVALID as never;
```